

Capstone Project Phase B

25-2-D-16

**Attenglish: English Learning Mobile App for ADHD
Pupils**



Attenglish
English learning app for ADHD pupils

Developed By:

Kfir Amoyal 205710411

Omri Heit 312333925

Academic Advisor:

Dr. Natali Levi

[GitHub Library](#)

Table of Contents

1. INTRODUCTION.....	3
2. LITERATURE REVIEW SUMMARY.....	4
3. GENERAL DESCRIPTION.....	5
4. DESCRIPTION OF THE DEVELOPMENT PROCESS.....	7
4.1. SYSTEM SOLUTION OVERVIEW.....	7
4.2. LEARNING ALGORITHMS AND PEDAGOGICAL LOGIC.....	7
4.3. USER MANAGEMENT AND ROLE-BASED ARCHITECTURE.....	7
4.4. AI TOOLS AND INTELLIGENT COMPONENTS.....	8
4.5. SYSTEM ARCHITECTURE.....	9
4.6. PACKAGE DIAGRAM.....	10
4.7. USER INTERACTION SEQUENCE.....	11
5. TOOLS AND STAKEHOLDER INTERFACE.....	12
5.1. INTERVIEWS SUMMARY.....	12
5.1.1. UI/UX AND VISUAL IDENTITY.....	12
5.1.2. FUNCTIONAL FEATURES AND PEDAGOGICAL REQUIREMENTS.....	12
5.1.3. GAMIFICATION AND PUPIL EXPERIENCE.....	13
6. CHALLENGES AND SOLUTIONS.....	13
7. RESULTS.....	14
7.1 SYSTEM OVERVIEW AND IMPLEMENTATION OUTCOMES.....	14
7.2 STAKEHOLDER FEEDBACK - EDUCATIONAL PERSPECTIVE.....	14
8. DECISION-MAKING AND HANDLING CHALLENGES.....	14
9. CONCLUSIONS AND LESSONS LEARNED.....	15
8.1. CONCLUSIONS.....	15
8.2. FUTURE WORK.....	15
10. PROJECT METRICS EVALUATION.....	16
10.1. QA EVALUATION.....	16
11. APPENDIX.....	21
11.1. ATTENGLISH - USER GUIDE.....	21
11.2. ATTENGLISH - DEVELOPER MAINTENANCE GUIDE.....	33

Abstract

Attention-Deficit/Hyperactivity Disorder (ADHD) is increasingly prevalent in classrooms worldwide, posing significant challenges to both students and educators [1]. These challenges stem from core ADHD symptoms - impulsiveness, inattention, and hyperactivity - that interfere with traditional methods of language acquisition. Today, ADHD pupils are typically supported through generalized accommodations such as extended time, reduced workloads, or individual assistance. While these approaches provide limited support, they depend heavily on teacher availability and lack personalization, real-time feedback, and continuity beyond the classroom, leaving language learning fragmented and insufficiently adapted to ADHD cognitive profiles.

Attenglish is a specialized mobile application designed to meet the unique needs of ADHD learners in elementary to high-school settings. By leveraging interactive modules, adaptive pacing, and evidence-based teaching strategies, the app seeks to boost engagement, bolster linguistic confidence, and provide real-time feedback to pupils, teachers, and parents. Attenglish offers not only effective language instruction, but also the tools for continuous progress monitoring, facilitating tailor-made interventions. As part of a broader inclusive education approach, this book presents how Attenglish addresses core ADHD learning gaps, lays out the engineering and testing processes, and positions the technology as a vital tool for teachers aiming to support ADHD pupils in mastering English.

1. Introduction

Learning English as a second or foreign language can be complicated for most lower-education pupils, but the process becomes particularly challenging for K-12 pupils¹ diagnosed with ADHD. Research indicates that children with ADHD often experience significant delays in reading proficiency, with approximately 30% showing notable challenges in reading and writing. These difficulties stem from core issues in sustaining focus, decoding language structures, and maintaining motivation over time.

While many existing language-learning platforms offer gamified lessons and interactive modules, few specifically address the unique challenges faced by ADHD students. Studies have shown that game-based learning can benefit students with ADHD by enhancing engagement and focus [3]. However, such strategies often lack adaptive technology capable of capturing data on attention spans, task completion rates, and progress markers.

Attenglish, the mobile application central to this project, integrates core elements of ADHD-friendly instruction - immediate AI-powered feedback, gamified interactions, and systematic skill reinforcement - into a cohesive ecosystem. The program employs branching pathways to accommodate different pacing speeds, frequent pop-up reminders to maintain attention, and brief, skill-specific games designed to enhance both vocabulary and grammatical fluency. These features are supported by AI-based data-tracking mechanisms that allow educators to identify stumbling blocks swiftly and provide targeted guidance.

Parents stand to benefit significantly from Attenglish by gaining clearer insight into their children's language-learning trajectory. School administrators can leverage aggregate data for decision-making related to curriculum design, student placement, and resource allocation. Moreover, the seamless integration of Attenglish into established classroom routines has the potential to reduce the stigma often associated with ADHD learners, promoting a more inclusive environment where they can thrive.

This book is structured to document the practical realization of the Attenglish project during Phase B, with an emphasis on system development, implementation, and empirical evaluation:

¹ Pupils in grades between first and twelfth.

1. **Background and Context** presents a concise overview of ADHD-related learning challenges and the pedagogical foundations motivating the development of Attenglish. This chapter serves to contextualize the project without repeating an extensive literature review conducted in Phase A.
2. **System Development and Implementation** details the development process of the Attenglish system, including architectural design, implementation of core functionalities for teachers and students, integration of AI-based components, and key technical and pedagogical design decisions made throughout the development phase.
3. **System Testing and Evaluation** describes the testing activities that were carried out, including functional testing, usability evaluation, and pilot usage analysis. The focus of this chapter is on validated results, user feedback, and observed system behavior, rather than on a prospective or planned testing methodology.
4. **Findings and Lessons Learned** synthesizes the main outcomes derived from the development and testing phases, identifying system strengths, limitations, and insights that may inform future enhancements and continued research.

By weaving together technology, pedagogy, and research-based strategies, Attenglish aspires to offer an enriched learning landscape for ADHD pupils. Through its user-centered design and thorough testing framework, the project aligns with inclusive education principles, setting the stage for meaningful and sustained improvements in English language proficiency among ADHD learners.

2. Literature Review Summary

In Phase B, we will explicitly build upon the literature review conducted in Phase A (Chapter 2), which provided the theoretical and empirical foundation for the Attenglish project. The Phase A review examined ADHD, technology-enhanced learning, executive functioning, gamification, and the state of English education in Israel. In this phase, we will extend those findings and translate them into concrete design and implementation decisions.

First, based on the findings presented in **Phase A, Section 2.1 (Experiences of Students with ADHD Using 3D, Haptic-Enabled Virtual Reality)**, we will adopt design principles derived from immersive and interactive learning environments. That section demonstrated that immersive technologies increase engagement, presence, and motivation among pupils with ADHD, while also emphasizing the necessity of structured tasks and guided interaction to prevent distraction. In Phase B, we will apply these principles by designing tightly scaffolded, interaction-rich activities that maintain learner focus without overwhelming attention resources.

Second, drawing on **Phase A, Section 2.2 (Practical Strategies to Assist Children with ADHD in the Classroom)**, we will incorporate evidence-based instructional strategies such as flexible pacing, task chunking, multimodal instruction, movement breaks, and consistent teacher-parent communication. The literature reviewed in this section highlighted that these strategies lead to improved academic performance and behavioral regulation, reinforcing our approach of adapting the learning environment rather than treating ADHD as a deficit.

Third, informed by **Phase A, Section 2.3 (Gamified Mobile-Technology Interventions for ADHD Learners)**, we will ground Attenglish's learning design in self-determination theory, flow theory, and universal design for learning. That section showed that short, adaptive, feedback-rich, and learner-controlled activities are particularly effective in sustaining attention and supporting language acquisition for pupils with ADHD. These findings will directly guide the structure of lessons, gamification mechanics, and adaptive difficulty systems in Phase B.

Fourth, building on the review presented in **Phase A, Section 2.4 (Managing ADHD Symptoms in Children through Technology-Driven Serious Games)**, we will leverage design patterns shown to produce moderate to large improvements in executive functions such as working memory, sustained attention, inhibition, and cognitive flexibility. At the same time, that section identified a notable gap in research connecting serious-game interventions directly to language-learning outcomes. In Phase B, Attenglish will explicitly address this gap by embedding executive-function supports within authentic English language-learning tasks.

Fifth, as discussed in **Phase A, Section 2.5 (Executive-Function and Working-Memory Training Apps)**, we will integrate adaptive difficulty, immediate multisensory feedback, and adult coaching mechanisms into the core learning experience. The literature emphasized that executive-function gains are strongest when cognitive scaffolding is embedded within meaningful academic tasks rather than isolated training exercises, a principle that will shape the internal logic and pacing of the Attenglish platform.

Sixth, informed by **Phase A, Section 2.6 (Parent-Engagement and Home-School Data Loops)**, we will implement real-time, positively framed feedback mechanisms for parents and teachers. That section demonstrated that low-latency, bidirectional data sharing improves intervention timing, parental involvement, and consistency across learning environments, making these features central to the system's architecture in Phase B.

Finally, drawing on **Phase A, Section 2.7 (English Studies in the Israeli Education System)**, we will position Attenglish as a scalable, data-driven, and inclusive response to systemic challenges such as teacher shortages, unequal access to advanced English instruction, and the absence of continuous formative assessment. These contextual findings will guide our emphasis on analytics, personalization, and institutional integration.

Overall, by explicitly extending the findings of the Phase A literature review, we will ensure that the design principles of Attenglish—short adaptive learning units, gamification, executive-function scaffolding, real-time analytics, and strong teacher-parent integration—remain firmly grounded in established research. In Phase B, the literature will continue to validate both the necessity and the potential educational impact of a dedicated English-learning platform tailored specifically to pupils with ADHD.

3. General Description

Attenglish was developed as a dedicated digital learning system that translates an existing, proven pedagogical approach into an accessible technological solution. The system is based on the instructional methods and day-to-day classroom practices of its client, Keren Yerushalmi, an experienced English teacher, who developed a structured work model for teaching English to K-12 pupils with ADHD.

Rather than introducing a new teaching methodology, Attenglish digitizes this established work form, preserving its pedagogical principles while enhancing them with attention-aware technology. The system was designed to reduce cognitive overload, increase sustained engagement, and enable measurable progress in core language skills - reading, writing, listening, and speaking - while integrating seamlessly into existing school routines.

The system was implemented as a mobile-first application supported by a cloud-based backend and a complementary web portal. Its structure was modular and adaptive by design. Learning content was divided into short, focused micro-lessons that typically lasted no more than a few minutes, reflecting the limited attention spans and executive-function challenges common among ADHD learners. Each learning unit combined multimodal elements such as text, audio narration, images, and interactive tasks, allowing learners to process information through multiple sensory channels.

At the core of the system was an adaptive logic layer that monitored learner interactions, including task completion time, accuracy, retries, and engagement patterns. Based on this data, the system dynamically adjusted difficulty levels, scheduled review activities using spaced repetition, and triggered supportive interventions such as remediation tasks, rewards, or attention breaks. Gamification elements - points, badges, streaks, and visual progress indicators - were embedded throughout the experience to reinforce motivation and provide immediate feedback without introducing unnecessary distractions.

Attenglish was structured as a multi-role ecosystem serving several user groups. The primary users were pupils with ADHD from elementary through high school, who interacted with the mobile application during class or at home. For them, the interface emphasized simplicity, calm visual design, and predictable navigation to minimize overstimulation.

Teachers constituted the main professional user group and accessed the system through a web-based dashboard. This portal allowed them to create and adapt lessons, assign content, monitor individual and class-level progress, and receive alerts when students struggled or showed readiness to advance. The system was designed to integrate into teachers' existing workflows rather than replace them, functioning as a supportive instructional and monitoring tool.

Parents formed a secondary but important audience. Through limited-access views and notifications, they were able to follow their children's progress, receive positive milestone updates, and support learning at home without being overwhelmed by raw data.

Overall, Attenglish functioned as an adaptive, data-driven learning environment that connected students, teachers, and parents around a shared goal: enabling ADHD learners to build meaningful and sustained proficiency in English through structured, engaging, and inclusive digital instruction.

4. Description of the Development Process

4.1. System Solution Overview

The Attenglish solution was designed as an end-to-end digital learning environment that combines pedagogical structure, adaptive logic, and intelligent automation to support English acquisition for pupils with ADHD. The system integrates a cross-platform mobile application with cloud-based backend services, enabling real-time interaction between learners, educators, and parents. The solution emphasizes short learning cycles, immediate feedback, and consistent routines, which are essential for maintaining attention and motivation among the target audience.

From a functional perspective, the system was divided into clearly defined modules, including learning content delivery, user management, progress analytics, and AI-assisted adaptation. This modular design allowed independent evolution of features while preserving overall system cohesion. The solution was validated continuously through iterative development, ensuring that technical implementation remained aligned with educational goals.

4.2. Learning Algorithms and Pedagogical Logic

At the core of Attenglish lies a rule-based and data-informed learning logic rather than a purely opaque machine learning model. This decision was intentional, as transparency, predictability, and explainability are critical when designing systems for children and educational stakeholders.

The lesson flow algorithm structured content into micro-learning units, each consisting of a single learning objective. Progression between units followed a mastery-based approach. Learners advanced only after demonstrating sufficient understanding through short interactive exercises. Incorrect responses triggered immediate corrective feedback and optional reinforcement activities.

To support long-term retention, the system implemented spaced reinforcement logic. Learning items were scheduled for review based on learner performance, with stronger items appearing less frequently and weaker items resurfacing sooner. This scheduling was adjusted dynamically using simple performance metrics such as accuracy, response time, and repetition count. The algorithm ensured repetition without overwhelming the learner, maintaining a balance between challenge and confidence.

Engagement-related parameters were also considered. The system monitored session length and interaction frequency to avoid cognitive overload. If signs of fatigue were detected, such as repeated errors or delayed responses, the system recommended session termination or lighter activities.

4.3. User Management and Role-Based Architecture

Attenglish supported multiple user roles, each with distinct permissions and responsibilities. These roles included students, teachers, and administrators. User management was implemented using role-based access control (RBAC), ensuring that each user could access only the data and functionalities relevant to their role.

Students interacted primarily with learning content and received feedback, progress indicators, and motivational elements. Their data was stored securely and associated with anonymized identifiers where possible to protect privacy.

Teachers were provided with dashboards for lesson configuration, class-level monitoring, and individual progress analysis. They could assign lessons, observe performance trends, and identify learners who required intervention. The system was designed to support pedagogical decision-making rather than replace it.

Parents had limited, read-only access focused on high-level progress summaries and engagement indicators. This design choice aims to encourage parental involvement without increasing pressure on the learner.

Administrators were responsible for system configuration, user provisioning, and policy enforcement. All authentication processes followed secure standards, and sensitive operations were logged for accountability and auditing purposes.

4.4. AI Tools and Intelligent Components

AI in Attenglish was intentionally designed as an assistive layer rather than a central decision-maker. To support this approach, **Claude was selected as the primary AI tool** due to its strong alignment with educational use cases that require controlled reasoning, clarity, and adherence to structured instructions.

Claude demonstrated high reliability in following predefined pedagogical rules, making it suitable for generating feedback that is consistent, age-appropriate, and educationally sound. Unlike more generative or autonomous models, Claude's responses could be effectively constrained, ensuring that AI feedback supported learning objectives without introducing unexpected interpretations or excessive linguistic complexity.

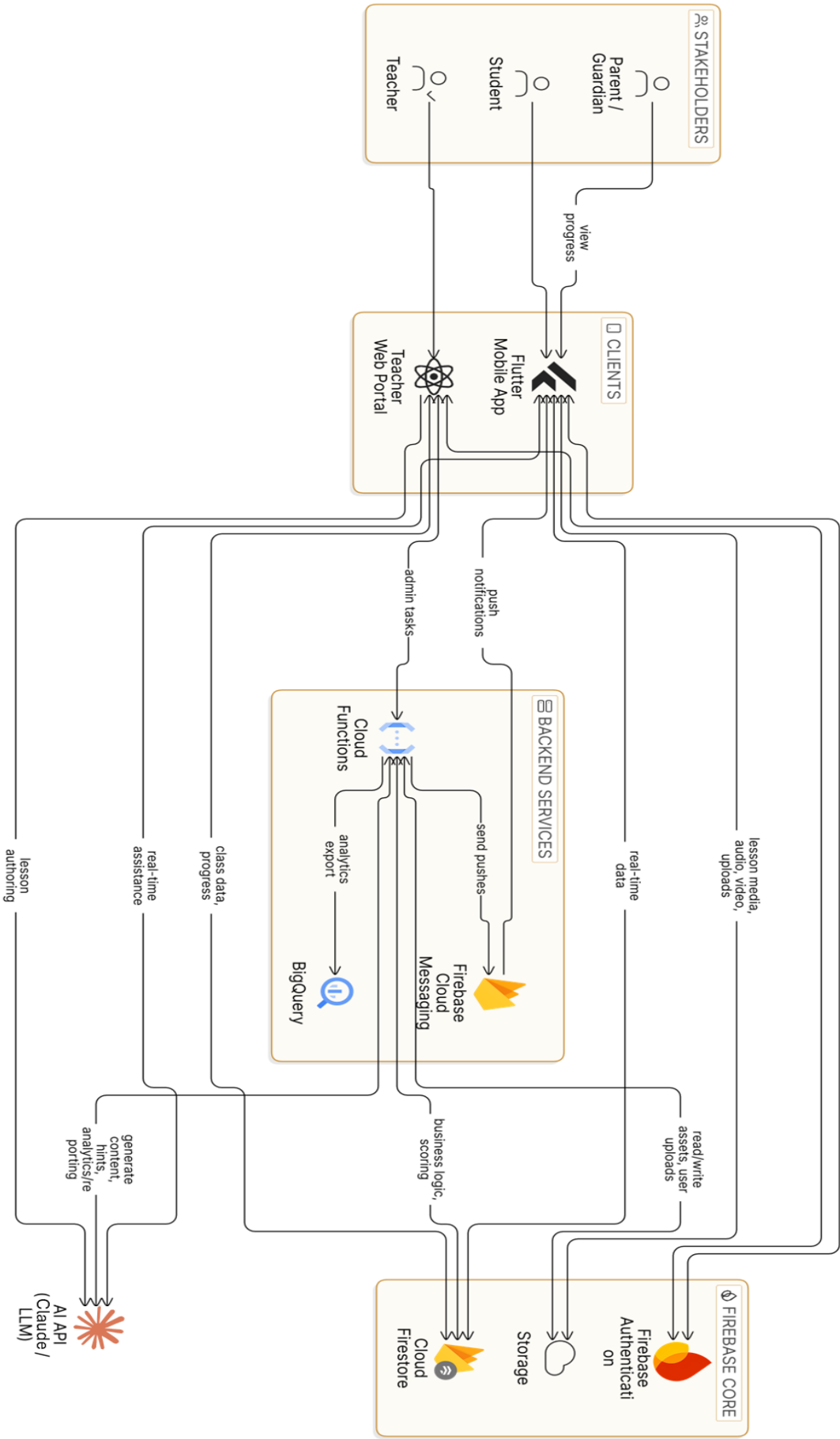
Natural language processing capabilities were used selectively to analyze learner input in open-ended exercises, such as short sentence construction or pronunciation-related feedback. Claude was leveraged to identify common error patterns rather than to perform full linguistic evaluation, reducing the risk of over-correction or misleading feedback. All outputs were filtered through rule-based pedagogical constraints to maintain consistency and instructional integrity.

For performance analytics, basic machine learning techniques were employed to cluster learners by difficulty patterns and engagement levels. Claude supported the interpretation and summarization of these analytics, translating raw patterns into teacher-facing insights presented through visual indicators and concise explanations rather than opaque predictions. This interpretability was a key factor in the model selection.

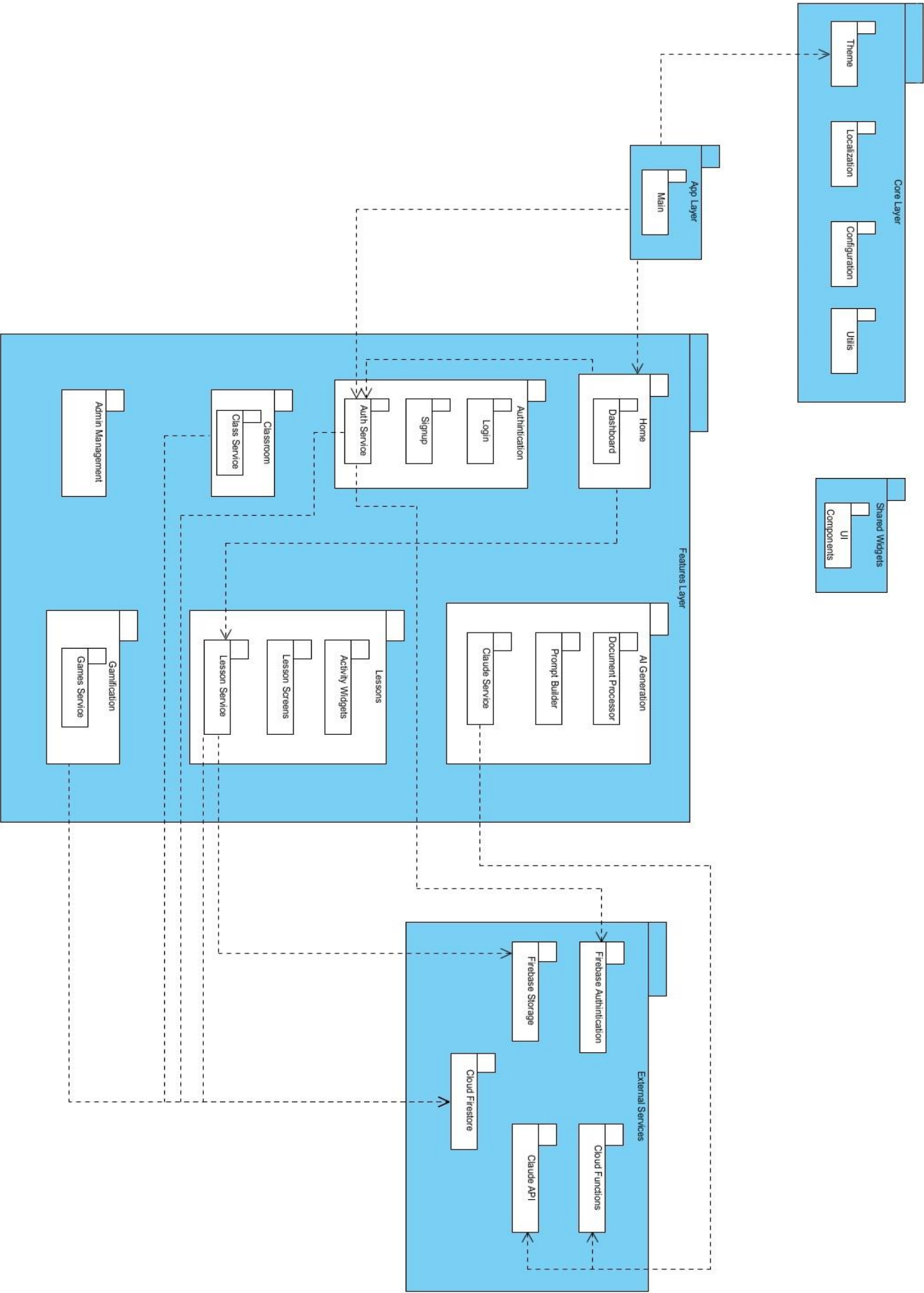
Additionally, Claude was used during content development to assist in generating exercise variants and distractors aligned with curriculum objectives. Its ability to maintain contextual coherence and follow explicit constraints reduced authoring time while preserving educational quality. All AI-generated content was reviewed and curated by educators prior to deployment.

Overall, Claude was chosen over alternative AI tools because it supported **structured assistance, interpretability, and controlled output**, aligning with Attenglish's goal of amplifying pedagogical structure, reducing manual overhead, and enabling adaptive behavior—while keeping educators and instructional intent firmly at the center of the system.

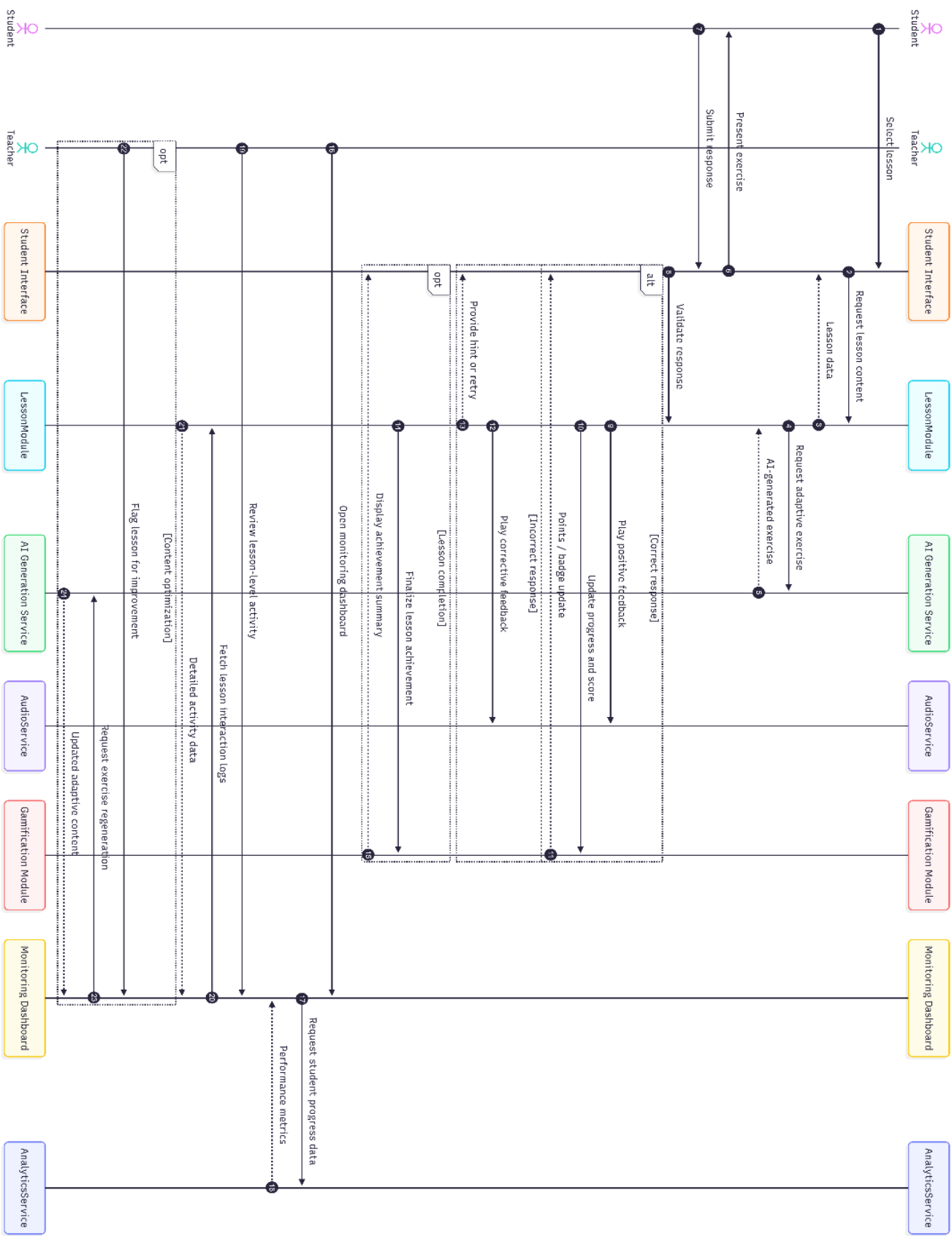
4.5. System Architecture



4.6. Package Diagram



4.7. User Interaction Sequence



5. Tools and Stakeholder Interface

The development of Attenglish relied on a modern software toolchain selected to support cross-platform deployment, rapid iteration, and long-term maintainability. A unified development framework enabled us to deliver a single application across multiple device types, which was essential for school environments with heterogeneous hardware. Cloud-based backend services were used to handle authentication, data storage, server-side logic, and media management, allowing the system to scale without complex infrastructure maintenance.

Version control and collaborative development practices were employed throughout the project to ensure traceability, controlled evolution of features, and safe experimentation. Design and prototyping tools were used to validate user-interface decisions, particularly those related to visual clarity, navigation simplicity, and accessibility for learners with attention difficulties. Testing tools supported both functional validation and regression prevention during iterative development.

A critical component of the development process was our continuous interface with the main stakeholder, Keren Yerushalmi. Rather than relying solely on requirement documents, we adopted a demonstration-based collaboration model. At regular intervals, we presented working features and partial implementations, allowing the stakeholder to evaluate them in the context of real classroom practice.

Feedback from these sessions directly influenced development priorities and design decisions. For example, the emphasis on micro-lessons, visual timers, positive reinforcement, and minimalistic interface elements were reinforced through stakeholder input. This close collaboration ensured that the system remained aligned with pedagogical realities and avoided divergence between engineering assumptions and educational needs.

5.1. Interviews Summary

5.1.1. UI/UX and Visual Identity

Keren initially praised the app's visuals as "wonderful," specifically noting that the large fonts are beneficial for students and teachers with ADHD. However, she provided specific critiques regarding the branding and interface:

- **Branding Adjustments:** She requested removing the British flag from the login screen and suggested making the brain logo more colorful and "catchy" to improve its visual appeal.
- **Teacher Dashboard Aesthetics:** Keren emphasized that because teachers are often tired themselves of "over-reading", the dashboard should be "happy," friendly, and cool rather than just plain colors. She suggested incorporating the new Attenglish logo into the background or side of the interface.
- **Layout:** She pointed out that some text input fields were too wide and needed to be narrowed for a better user experience.

5.1.2. Functional Features and Pedagogical Requirements

The discussions covered several key functional components designed to assist teachers in lesson preparation:

- **AI Lesson Generator:** Keren inquired if teachers could generate lessons by simply typing a topic (e.g., adjectives) in one line rather than always uploading a file, though the current system primarily relies on file uploads.

- **Question Types:** She approved of the existing Multiple Choice, True/False, and Drag & Drop formats. She strongly recommended adding "Completion" (cloze) questions and "Open-ended/Thinking" questions to build the foundational skills students need for future matriculation exams.
- **Content Constraints:** For the target audience elementary school pupils, she advised limiting text length to approximately 100 words to keep the content focused and manageable.
- **Output Options:** A significant request was the ability for teachers to export worksheets as PDFs or print them directly from the browser for classroom use.
- **Student Management:** The system currently supports importing students via Excel files, which Keren accepted, though she questioned if it could eventually integrate with cloud platforms like Google Classroom.

5.1.3. Gamification and Pupil Experience

To enhance pupil engagement, we showcased several gamification elements:

- **Leveling System:** pupils progress through levels, represented by an owl avatar that "hatches" and grows as they complete tasks.
- **Engagement Tools:** The app includes learning "streaks" and digital "tags" as rewards for achievements.
- **Interactive Activities:** One featured activity involves students describing an image, which the AI agent then evaluates for accuracy. While Keren suggested an AI image generator for teachers to create custom prompts, we did note this might be limited by current API costs and budget constraints.

6. Challenges and Solutions

The project introduced a range of analytical, engineering, and technical challenges, many of which stemmed from the intersection of software design and ADHD-focused pedagogy.

One central challenge was balancing engagement with overstimulation. While gamified elements are effective for increasing motivation, excessive visual or auditory stimuli can negatively affect learners with attention difficulties. We addressed this challenge by deliberately limiting the intensity and frequency of animations, sounds, and visual effects. Gamification elements were designed to be brief, predictable, and supportive rather than distracting.

Another major challenge involved the design of adaptive learning mechanisms. Although advanced machine-learning approaches were considered, they posed concerns related to transparency, explainability, and computational cost, particularly in constrained school environments. We therefore adopted rule-based adaptive strategies that adjusted content repetition and difficulty based on observable performance patterns. This approach provided meaningful personalization while remaining reliable and interpretable for educators.

From a data-management perspective, the system needed to support frequent user interactions without compromising performance or data integrity. We addressed this by separating short-term progress indicators from long-term historical data, enabling efficient real-time feedback while preserving analytical depth. This design also facilitated offline operation and later synchronization.

Supporting offline usage was itself a significant technical challenge. Many school-issued devices operate with limited or unstable connectivity. We implemented local data caching and synchronization logic that allowed learners to continue working offline and safely merge their progress once connectivity was restored.

7. Results

7.1 System Overview and Implementation Outcomes

At the conclusion of Phase B, the Attenglish system was fully implemented and evaluated in alignment with the objectives, requirements, and testing plan defined in Phase A. The resulting product is a fully operational, cross-platform mobile application designed to support English language learning for pupils with ADHD and ADD from elementary through high school.

All primary use cases specified in Phase A were successfully implemented and validated. These included structured student learning flows, short and goal-oriented exercises, adaptive practice and reinforcement mechanisms, progress tracking, and role-based access for teachers and parents. Functional testing confirmed that the system operated reliably across supported platforms and maintained consistent behavior under typical usage conditions.

From a pedagogical standpoint, the system reflected ADHD-sensitive instructional principles. Learning activities were intentionally designed to be concise, visually engaging, and clearly focused on single learning objectives in order to reduce cognitive load and support sustained attention. The integration of multimodal content - including text, audio, and interactive elements - addressed diverse learning styles and reinforced comprehension. Gamification elements such as points, progress indicators, and positive feedback mechanisms were incorporated in a controlled manner, supporting motivation and engagement without detracting from educational goals.

The system's adaptive behavior further contributed to its effectiveness. Exercises responded to student performance by enabling repetition, reinforcement, and gradual difficulty adjustment. This supported individualized learning paths and helped promote consistent experiences of success, which are particularly critical for learners with attention-related challenges.

7.2 Stakeholder Feedback - Educational Perspective

According to Keren, Attenglish aligned well with real-world classroom practices and addressed several common challenges in English instruction for students with ADHD, including maintaining attention, managing pacing, and providing immediate feedback. She emphasized the value of short, clearly structured exercises and noted that the system's design supports classroom management rather than disrupting it. The availability of teacher-oriented monitoring tools was perceived as particularly beneficial, enabling informed pedagogical decisions based on concrete progress data rather than intuition alone.

Overall, the stakeholder feedback indicated that Attenglish was not perceived merely as a technological product, but as a meaningful educational intervention capable of supporting both learning outcomes and teaching workflows.

8. Decision-Making and Handling Challenges

Throughout the project, our decision-making process was guided by a consistent prioritization of educational effectiveness, usability, and reliability over technical novelty. When multiple implementation options were available, we favored solutions that were transparent, predictable, and robust in real-world school settings.

For instance, we deliberately chose deterministic adaptive mechanisms over opaque AI-driven models to ensure that system behavior could be understood and trusted by teachers. Similarly, we repeatedly simplified user flows when they risked increasing cognitive load, even when this meant reducing the overall number of features.

These decisions were informed by ADHD-specific cognitive considerations, stakeholder feedback, time constraints, and the operational limitations of school infrastructure. This disciplined approach helped maintain focus on the project's core objectives.

9. *Conclusions and Lessons Learned*

Reflecting on the development phase, we believe that our development approach was largely effective, particularly our emphasis on early architectural planning and continuous stakeholder involvement. These practices significantly reduced misalignment and rework.

Nevertheless, several lessons emerged. In future iterations, we would invest earlier in comprehensive automated testing and performance monitoring, especially for offline and synchronization scenarios. We would also initiate limited classroom pilots sooner to complement qualitative feedback with quantitative usage data.

A key lesson was the importance of restraint. Features that were carefully designed, minimal, and pedagogically justified consistently produced better outcomes than more complex alternatives. This insight will strongly influence our approach to future educational software projects.

8.1 Conclusions

Based on the results of functional testing, stakeholder feedback, and project metrics evaluation, it can be concluded that Attenglish successfully fulfilled its intended role. The project achieved its dual objective of delivering a robust technological solution while addressing a genuine educational need.

Attenglish demonstrates that thoughtfully designed educational technology, when grounded in pedagogical principles and real classroom requirements, can effectively support inclusive learning. The system provides value not only to students with ADHD, but also to teachers and parents by enabling structured learning, informed monitoring, and positive reinforcement.

8.2 Future Work

While the current implementation met the project's objectives, several directions for future development and research were identified:

- Expansion of learning content to cover additional language domains, such as advanced grammar, extended writing tasks, and speaking-focused activities.
- Integration of more advanced analytics to identify long-term learning patterns and support early detection of learning difficulties.
- Exploration of AI-driven personalization features, such as more fine-grained adaptation of difficulty levels and feedback styles, while maintaining pedagogical control.
- Longitudinal studies involving a larger population of students to quantitatively measure learning outcomes over extended periods.
- Enhanced integration with school systems to support broader adoption within formal educational frameworks.

These future directions highlight Attenglish's potential as a scalable and extensible platform, providing a solid foundation for continued development and academic investigation.

10. Project Metrics Evaluation

The evaluation of the Attenglish project in Phase B was conducted in alignment with the success criteria and expected achievements defined in Phase A. These criteria focused on technical performance, usability for pupils with ADHD/ADD, educational effectiveness, and stakeholder satisfaction. Overall, the findings indicate that the system largely fulfilled its intended objectives and demonstrated readiness for broader pedagogical evaluation.

From a technical perspective, the application met the predefined performance requirements. The system operated smoothly across the targeted mobile platforms, with stable responsiveness and acceptable loading times on standard school devices. No critical crashes were observed during testing, and the adaptive mechanisms successfully logged user interactions such as task completion time, retries, and skipped items. This confirmed that the core personalization logic functioned as intended and was capable of dynamically adjusting learning content based on user behavior.

In terms of usability, the application proved accessible and intuitive for the target audience. Pupils with ADHD were able to navigate the structured learning paths and interactive exercises independently following minimal onboarding. Design choices emphasizing short lessons, visual cues, and immediate feedback contributed to sustained engagement and reduced cognitive overload. Informal usage observations indicated a clear preference for the application compared to traditional learning tools, supporting the usability goals outlined in Phase A.

Educational effectiveness was evaluated through usage patterns and performance indicators rather than long-term outcome measures. Students demonstrated improved task efficiency, reflected in shorter completion times and fewer retries across repeated exercises. Engagement metrics, including consistent weekly use, suggested that the gamified elements and adaptive difficulty contributed positively to motivation. While comprehensive CEFR-aligned pre- and post-testing requires extended deployment, the observed trends provide initial evidence that the system supports skill development in vocabulary, grammar, and comprehension.

Stakeholder satisfaction was also a key evaluation dimension. Feedback from the primary stakeholder, Keren Yerushalmi, and other English teachers indicated that the teacher dashboard was informative and practical. Educators highlighted the value of clear progress visualization, individual performance insights, and the ability to assign targeted learning modules. These features supported lesson planning and enabled more focused reinforcement of students' weaker areas.

In summary, the Phase B evaluation demonstrates that Attenglish successfully meets the majority of its defined success metrics. The system delivers a technically stable, usable, and pedagogically aligned learning environment tailored to neurodiverse learners. Although long-term educational impact requires further study, the current results confirm that the project has achieved its core objectives and provides a solid foundation for broader assessment and iterative enhancement.

10.1. QA Evaluation

#	Actor	Core Use-Cases to Validate	Testing Flows	Outcomes
1	Student	Login / Logout	Path flow: Login → open classroom → start task → exit	The application launched successfully and allowed navigation to the main classroom screen. Basic entry and exit flows worked as expected. Full

				authentication scenarios were not implemented in Phase B.
2	Student	Access completed lessons / stories	Offline mode: start a cached learning game	Cached content was accessible once loaded. Full offline persistence across sessions was not fully supported and was partially validated.
3	Student	View multimodal content	AI vision: upload image, receive description	The AI service successfully returned generated descriptive content. The response was displayed correctly within the application flow.
4	Student	Perform interactive activities	Complete short focused activity	Interactive components responded correctly to user input, and activity completion did not cause runtime errors.
5	Student	Use visual timer for timed tasks	Start task with timer → timer countdown	The visual timer displayed and counted down as expected during tasks, supporting time-focused activities.
6	Student	Receive feedback and rewards	Complete activity → receive feedback	Immediate feedback was presented upon activity completion. Gamified reinforcement elements were displayed correctly.
7	Teacher	Create and manage lessons	Create lesson → assign to classroom	Lesson creation logic existed at the data level. Full UI-based management was not fully implemented in Phase B.
8	Teacher	Monitor student progress	View progress dashboard	Progress indicators were available in basic form. Detailed analytics and historical tracking were limited.
9	System	AI-generated exercises	Trigger AI generation → receive exercise	AI-generated exercises were successfully produced and integrated into the lesson flow using the AI service layer.
10	System	App stability	Launch app → perform core flows	No critical crashes were observed during testing. The application remained stable throughout the tested flows.
11	Teacher	Classroom management	Create classroom → add students → remove student → delete classroom	Classrooms are created with name, grade, and school. Students are assigned and removed correctly. Deletion requires confirmation and removes all associations.

12	Teacher	Bulk student import	Upload Excel file → validate data → import students to class	Students are created from Excel/CSV and appear in the classroom roster. Invalid rows display error messages. File type validation rejects unsupported formats.
13	Teacher	AI generation full flow	Paste text → configure grade/difficulty/activity counts → generate → review → accept	Claude AI generates activities matching the requested configuration. Teacher can preview, edit, and delete individual activities before accepting. Word count validation enforces 50-5,000 word range.
14	Teacher	Lesson publishing flow	Create lesson (draft) → add content and activities → publish	Draft lessons are saved but not visible to students. After publishing, lessons appear in the student's mission list. Status is correctly reflected in the Content Library.
15	Teacher	Edit and delete lessons	Open existing lesson → edit title/content → save; Delete lesson → confirm	Lesson updates persist in Firestore. Associated files (audio, images) are deleted from Storage when lesson is removed. Deleted lessons no longer appear for students.
16	Teacher	Activity type creation	Create one of each activity type manually (Multiple Choice, Fill in Blank, True/False, Drag & Drop, Image Description, Cloze, Open-ended)	Each activity type form validates required fields. Activities are saved correctly to the lesson. Points are configurable per activity.
17	Student	Gamification - XP and levels	Complete activities → earn XP → accumulate to level up	XP is awarded per activity type (True/False: 5, Multiple Choice/Fill Blank: 10, Drag & Drop: 15). Lesson completion awards 50 XP. Perfect score bonus: 25 XP. Level progresses from Hatchling (1-2) through to Legendary Owl (20+).
18	Student	Daily streaks	Complete lesson day 1 → complete lesson day 2 → skip day 3 → return day 4	Streak increments on consecutive days. Streak bonus (20 XP) awarded daily. Streak resets to zero after a missed day. Current and longest streak are tracked separately.
19	Student	Drag & Drop activity	Start drag-drop activity → drag items	Correct matches are accepted and highlighted. Incorrect matches provide

			to correct matches → submit	visual feedback. Score reflects the number of correct pairs.
20	Student	AI-evaluated activities	Complete open-ended / cloze / image description → submit free-text answer	Claude AI evaluates the student's free-text response against the sample answer. Returns a score and qualitative feedback. Score is applied to lesson progress.
21	Student	Student dashboard components	Login → view progress bar, streak calendar, classroom info, missions	Dashboard displays current level/XP with progress bar, last 7 days streak calendar, classroom and teacher name, and up to 3 lesson mission cards.
22	Student	View all tasks	Click "View All Tasks" → browse all assigned lessons	All assigned lessons are listed. Completed lessons show "DONE" badge. New lessons show "NEW" badge. Tapping a lesson opens the lesson viewer.
23	Admin	Manage teachers	Admin login → create teacher account → verify teacher login	Teacher account is created in Firebase Auth and Firestore with correct role. Teacher can log in and access the teacher dashboard with all features.
24	Admin	Manage students	Admin login → create student → assign to class	Student account is created with all required fields. Student appears in the selected classroom roster and can log in to the student dashboard.
25	System	Multi-language support	Toggle language English → Hebrew → English	All UI text switches language correctly. Layout changes to RTL for Hebrew and back to LTR for English. Language preference persists after page reload.
26	System	Role-based access control	Login as student / teacher / admin → verify different dashboards	Each role sees only their permitted dashboard and features. Students cannot access teacher or admin screens. Navigation guards enforce role boundaries.
27	System	Tutorial system	First login → tutorial auto-appears → navigate Next/Back → finish; Replay via Help button	Tutorial shows automatically on first login for each role. Steps are role-specific. Skip/Back/Next/Finish buttons work. Completion is saved in SharedPreferences. Help button replays the tutorial.

28	System	Input validation and error handling	Submit empty login form → enter invalid email → enter wrong password → submit empty lesson form	Clear error messages displayed for each validation failure. Firebase auth errors show user-friendly messages. Forms prevent submission with missing required fields. Network errors are caught and displayed.
----	--------	-------------------------------------	---	---

11. Appendix

11.1. Attenglish - User Guide

Table of Contents

- [Introduction](#)
 - [Getting Started](#)
 - [Teacher Guide](#)
 - [Student Guide](#)
 - [Tutorial System](#)
-

Introduction

Attenglish is an English learning platform designed specifically for ADHD students (ages 7-12). It provides teachers with AI-powered tools to create engaging, interactive lessons and gives students a gamified learning experience.

Access the app: <https://attenglish-dev.web.app>

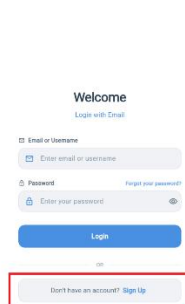
Key Features: - AI-powered activity generation using Claude AI - 8 interactive activity types - Gamification system with XP, levels, streaks, and achievements - Progress tracking and analytics for teachers - Multi-language support (English and Hebrew) - ADHD-friendly design with clear visuals and reduced clutter

Getting Started

Creating an Account

Teacher Registration

1. Open the app and click **Sign Up**



2. Navigate to the **Admin Sign Up** option

3. Fill in: email, password, full name, username, school name, city

4. Submit the form to create your account

Student Registration

1. Open the app and click **Sign Up**

2. Fill in: email, password, full name, username, school name, city, grade level

3. Submit the form to create your account

Note: Teachers can also create student accounts in bulk via the Admin panel or Excel import.

Logging In

1. Enter your **username or email**

2. Enter your **password**

3. Click **Log In**

Language Toggle

- Click the **language toggle** button (top-right corner) to switch between English and Hebrew
- The entire interface updates in real-time



Teacher Guide

Dashboard Overview

After logging in, the teacher dashboard displays 5 main cards:

Card	Description
AI Generator	Create activities automatically using Claude AI
My Classrooms	Manage classrooms and student rosters
Student Progress	View performance charts and analytics
Create Lesson	Build new lessons with content and activities
Content Library	Browse and manage all existing lessons

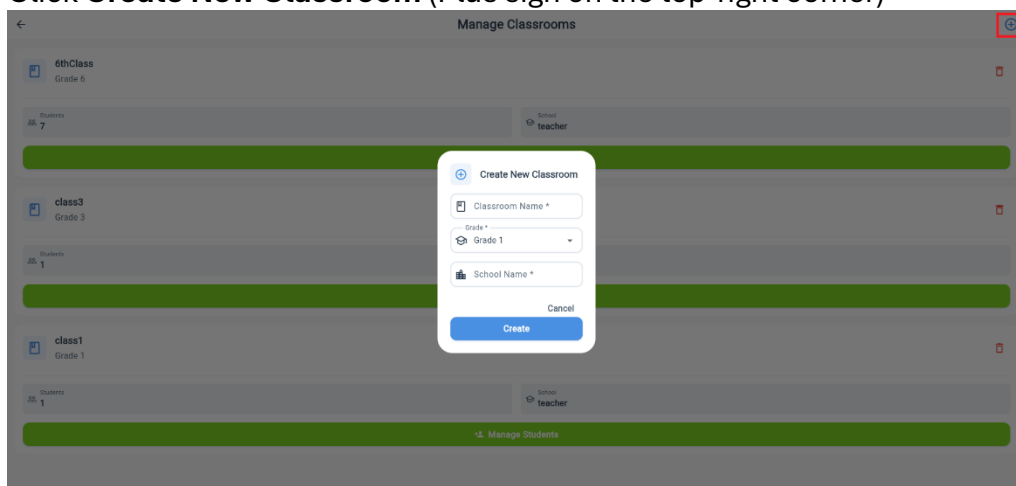
Classroom Management

Creating a New Classroom

1. Click **My Classrooms** from the dashboard



2. Click **Create New Classroom** (Plus sign on the top-right corner)



3. Enter: classroom name, grade level (1-12), school name
4. Click **Create**

Adding Students to a Classroom

- **Individual:** Click on a classroom, then click **Add Student** and fill in student details
- **Bulk Import:** Click **Import from Excel** to upload a spreadsheet of students
 - Excel format: columns for name, email, username, password, grade

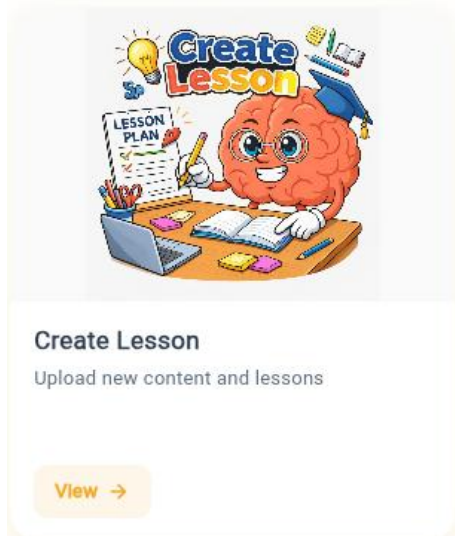
Managing Classrooms

- **Edit** classroom name, grade, or school
- **Delete** a classroom (with confirmation dialog)
- **Remove** individual students from a classroom

Creating Lessons

Step-by-Step Lesson Creation

1. Click **Create Lesson** from the dashboard



2. Fill in the lesson info:
 - **Title** (required)
 - **Description** (optional)
 - **Classroom** - select which class this lesson is for (required)

- **Grade Level** (required)
 - **Difficulty** - 1 to 5 (required)
3. Add the **lesson content**:
 - Paste or type the English text/story
 - Upload **audio** (optional) - for listening exercises
 - Upload **images** (optional) - visual aids
 4. Add **activities** (see Activity Types below)
 5. Choose to **Save as Draft** or **Publish**

Note: Only published lessons are visible to students.

Create New Lesson

Create a new lesson for your students

T

0/100

D

0/300

C

Classroom *

▼

G

Grade Level *

▼

D

Difficulty *

▼

T

0/100

Lesson Content *

A

Audio File

Upload an audio file (MP3, WAV, M4A) - Max 10MB

A Upload Audio

I

Images

0/10

Upload images (JPG, PNG, GIF) - Max 10 images, 5MB each

I Upload Images

A Lesson Activities

0 activities

Add interactive exercises to help students practice (optional)

+ Add Activity

Cancel

S Save as Draft

P Publish

Activity Types

Teachers can add 8 types of activities to a lesson:

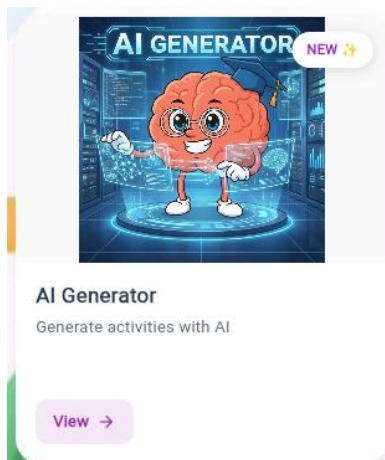
- 1. Multiple Choice** - Write a question - Provide 4 answer options - Select the correct answer - Points: configurable (default 10)
 - 2. Fill in the Blank** - Write a sentence with a blank space - Enter the correct word/phrase - Answers are checked case-insensitively - Points: configurable (default 10)
 - 3. True/False** - Write a statement - Select whether it is true or false - Points: configurable (default 10)
 - 4. Drag & Drop (Matching)** - Create left-side items (e.g., English words) - Create right-side items (e.g., definitions or translations) - Map the correct pairs - Students drag to match - Points: configurable (default 10)
 - 5. Image Description (AI-Evaluated)** - Upload 1-3 images - Provide a sample answer for reference - Students write a description of the image(s) - Claude AI evaluates the response - Points: configurable (default 10)
 - 6. Cloze Question (AI-Evaluated)** - Provide a story/passage context - Create a fill-in-the-blank from the story - Enter the expected answer - Claude AI scores the response - Points: configurable (default 10)
 - 7. Open-ended Question (AI-Evaluated)** - Provide a story/passage context - Write a reading comprehension question - Provide a sample answer - Claude AI evaluates the student's response - Points: configurable (default 10)
 - 8. AI Generated (Bulk)** - Use the AI Generator to create multiple activities at once (see below)
-

AI Activity Generation

The AI Generator uses Claude AI to automatically create activities from any English text.

How to Use

1. Click **AI Generator** from the dashboard



2. **Write\Paste your text** in the text box (50-5,000 words)

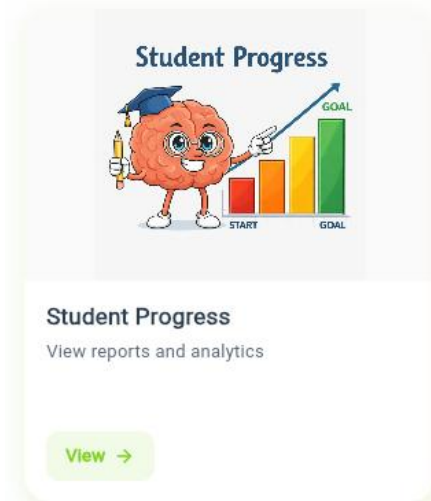
3. **Configure settings:**
 - Select **Classroom**
 - Choose **Grade Level** (1st-12th)
 - Set **Difficulty** (Beginner, Intermediate, Advanced)
4. **Set activity counts** for each type:
 - Multiple Choice (default: 5)
 - Fill in the Blank (default: 3)
 - True/False (default: 3)
 - Drag & Drop (default: 2)
 - Image Description (default: 0)
 - Cloze Questions (default: 0)
 - Open-ended Questions (default: 0)
 - Maximum total: 50 activities
5. Click **Generate**
6. Wait for AI to process (loading indicator shown)
7. **Review the generated activities:**
 - Preview each activity
 - Edit any activity details
 - Delete unwanted activities
8. Click **Accept** to add them to a lesson

The screenshot displays the 'AI Activity Generator' interface. At the top, there's a header with 'AI Activity Generator' and a 'Generate' button. Below the header, a blue bar contains the instruction: 'Paste English text (50-5000 words) and I'll generate activities for you!'. The main area is divided into two sections. The top section is for 'English Text' with a large text input area. The bottom section is titled 'Activity Settings' and includes a dropdown for 'Grade Level' (set to '5th Grade') and a button for 'Difficulty' (set to 'Intermediate'). Below these are two columns of activity types with their respective counts and plus-minus buttons to adjust them: Multiple Choice (5), Fill in the Blank (3), True/False (3), Drag & Drop (2), Image Description (0), Cloze Questions (0), Open-ended Questions (0), and a 'Total Activities' count of 13. At the bottom, there's a blue bar with a 'Generate Activities' button.

Student Progress

Viewing Class Performance

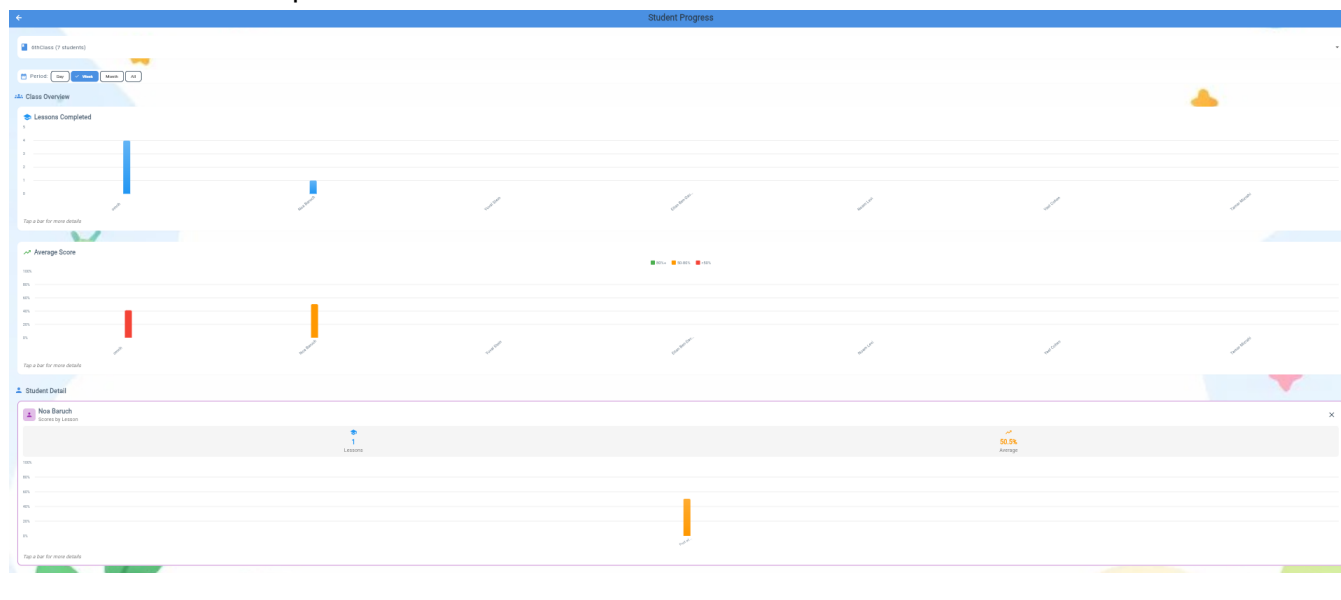
1. Click **Student Progress** from the dashboard



2. Select a **Classroom** from the dropdown
3. Choose a **Time Range**: Week, Month, or All-time
4. View two charts:
 - **Lessons Completed** - bar chart showing lessons completed per student
 - **Average Scores** - bar chart showing average performance per student

Viewing Individual Student Details

1. Click on a student's name or bar in the chart
2. View a detailed chart showing their score for each lesson
3. Track their improvement over time



Student Guide

Dashboard Overview

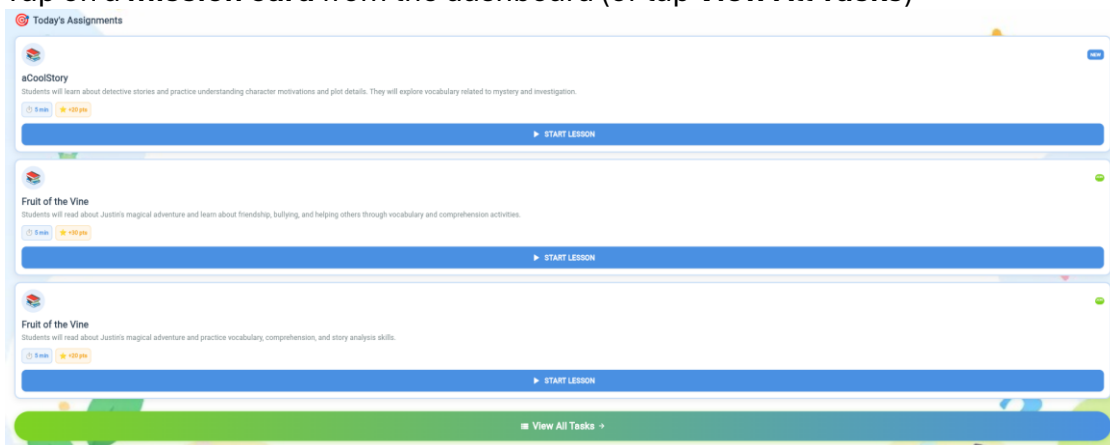
After logging in, the student dashboard shows:

Section	Description
Greeting	Personalized hello with the Atti mascot companion
Progress Bar	Current XP and level progress
Streak Calendar	Last 7 days of activity, current streak count
Classroom Info	Your classroom and teacher name
Today's Missions	Up to 3 lessons to complete
View All Tasks	See all available lessons

Completing Lessons

Starting a Lesson

1. Tap on a **mission card** from the dashboard (or tap **View All Tasks**)



2. Read the lesson content (story/text)
3. Listen to audio if available (tap the play button)
4. View any images included
5. Tap **Start Activities** to begin

Navigating Activities

- Activities are presented **one at a time**
- A **progress bar** at the top shows how far you are
- “**Activity X of N**” counter shows current position
- Your **running score** is displayed at the top right

Answering Activities

Activity Type	How to Answer
Multiple Choice	Tap one of the 4 options
Fill in the Blank	Type the missing word in the text box
True/False	Tap True or False

Activity Type	How to Answer
Drag & Drop	Drag items from the left and drop them on the matching item on the right
Image Description	Look at the image(s) and type your description in the text box, then tap Submit
Cloze	Read the story and type the missing word in the text box
Open-ended	Read the story and write your answer in the text box, then tap Submit

Finishing a Lesson

- After the last activity, tap **Finish Lesson**
- Your total score and XP earned will be displayed
- The lesson will be marked as completed on your dashboard

Gamification System

XP (Experience Points)

You earn XP for every activity you complete:

Action	XP Earned
Multiple Choice (correct)	10 XP
Fill in the Blank (correct)	10 XP
True/False (correct)	5 XP
Drag & Drop (correct)	15 XP
Lesson Completed	50 XP
Perfect Score Bonus	+25 XP
Daily Streak Bonus	20 XP

Levels

As you earn XP, you level up through owl tiers:

Level	Tier	Title
1-2	Hatchling	Just starting out
3-5	Fledgling	Growing learner
6-9	Young Owl	Getting confident
10-14	Wise Owl	Strong skills
15-19	Master Owl	Almost at the top
20+	Legendary Owl	English master

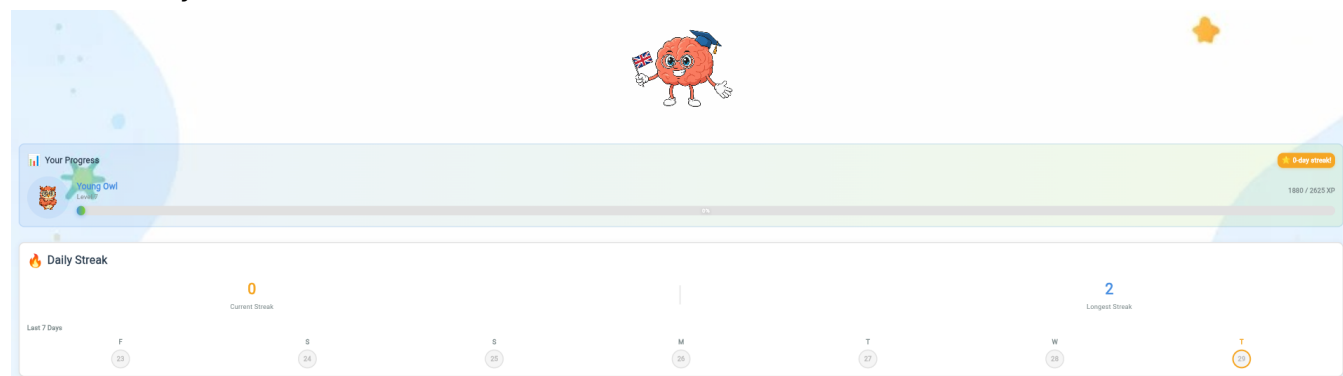
Daily Streaks

- Complete at least one lesson every day to build a streak
- Your current and longest streak are tracked
- Streak bonus: 20 XP per day you keep it going

- Missing a day resets your streak to zero

Achievements

- Unlock badges as you reach milestones
- View your achievements on the dashboard



Tutorial System

First-Time Users

- When you log in for the first time, a **step-by-step tutorial** automatically appears
- The tutorial walks you through all the main features of the app
- Navigate using **Next**, **Back**, or **Skip** buttons

Replaying the Tutorial

- Click the **Help** button (question mark icon) on the dashboard
- The tutorial will replay from the beginning

Note: Tutorials are different for students and teachers, showing relevant features for each role.

Platform Notes

- **Teachers:** The teacher portal is optimized for **web browsers** (desktop/laptop). If you open the app on a mobile phone, you will see a message directing you to use a desktop browser.
- **Students:** The app works on both **web and mobile** devices.

Troubleshooting

Issue	Solution
Can't log in	Check your username/email and password. Use the correct case.
Dark screen on login	Clear browser cache and refresh the page
AI generation taking too long	Wait up to 60 seconds. If it fails, try again with shorter text.
Activities not showing	Make sure the lesson is Published (not Draft)
Language not changing	Click the language toggle again; refresh if needed

Issue	Solution
Audio not playing	Check your device volume and browser audio permissions

11.2. Attenglish - Developer Maintenance Guide

Table of Contents

- [Project Overview](#)
- [Getting Started](#)
- [Architecture](#)
- [Firebase Configuration](#)
- [Cloud Functions](#)
- [AI Integration](#)
- [Key Services Reference](#)
- [Build & Deploy](#)
- [Dependencies](#)
- [Troubleshooting](#)

Project Overview

Tech Stack

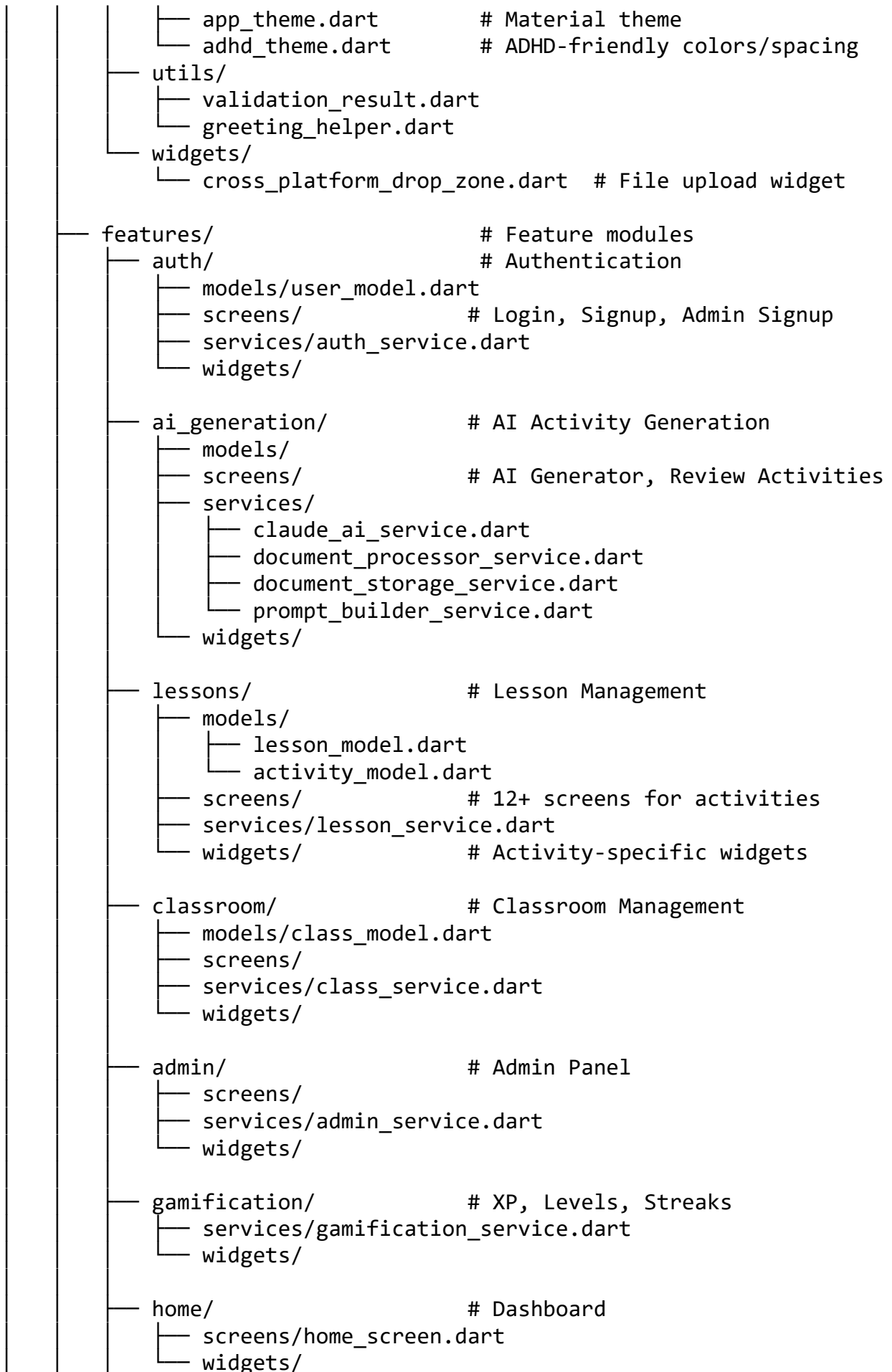
Layer	Technology
Frontend	Flutter (Dart)
Backend	Firebase (Firestore, Auth, Storage, Cloud Functions)
AI	Claude API (Anthropic) via Cloud Functions
Hosting	Firebase Hosting
State Management	StatefulWidgets + Streams + Singleton Services
Routing	Named routes (Navigator)
Charts	fl_chart

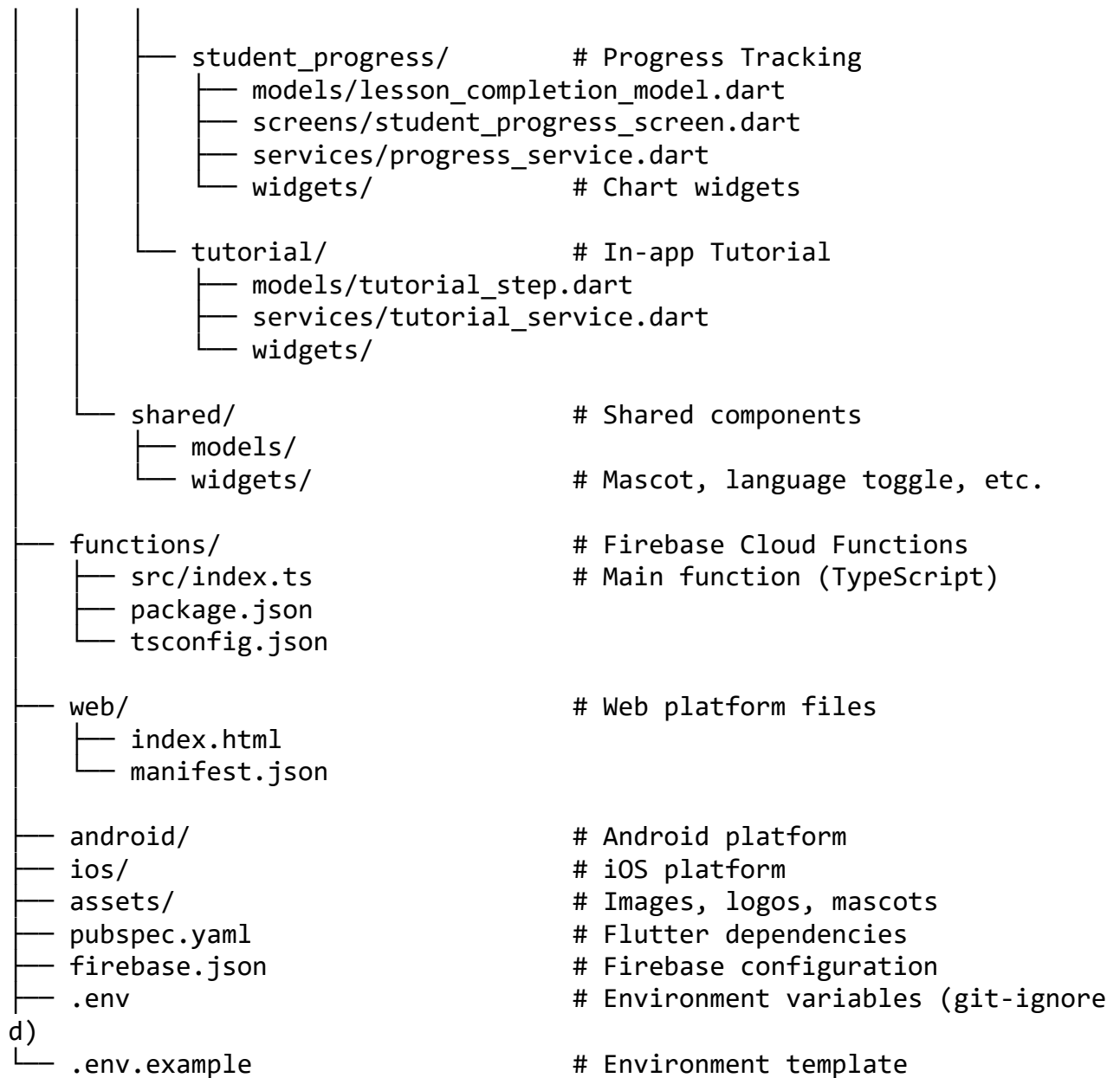
Architecture

The project follows a **Feature-based Architecture** where each feature is isolated in its own directory with models, screens, services, and widgets.

Directory Structure

```
attenglish/
├── lib/
│   ├── main.dart                # App entry point, routing, theme
│   ├── firebase_options.dart    # Auto-generated Firebase config
│   └── core/                    # Shared core functionality
│       ├── config/
│       │   └── ai_config.dart   # Claude AI configuration
│       ├── constants/
│       │   └── app_strings.dart # Localized text strings
│       ├── services/
│       │   └── localization_service.dart # i18n support
│       └── theme/
```





Getting Started

Prerequisites

Tool	Version	Purpose
Flutter SDK	^3.5.4	Frontend framework
Dart SDK	^3.5.4	Programming language (bundled with Flutter)
Node.js	20.x	Cloud Functions runtime
Firebase CLI	Latest	Firebase project management
Git	Latest	Version control

Setup Instructions

1. Clone the repository
`git clone <repository-url>`

```

cd attenglish/attenglish

# 2. Install Flutter dependencies
flutter pub get

# 3. Set up environment variables
cp .env.example .env
# Edit .env and add your ANTHROPIC_API_KEY

# 4. Install Cloud Functions dependencies
cd functions
npm install
cd ..

# 5. Run on web (Chrome)
flutter run -d chrome

# 6. Run on Android
flutter run -d <device-id>

```

Environment Variables

File: `.env` (must be created from `.env.example`)

`ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxx`

- The API key is loaded at startup via *flutter_dotenv*
- Used by *ClaudeAiService* to authenticate with the Claude API
- On web, API calls go through Cloud Functions (key passed securely)
- On mobile/desktop, direct API calls are made

Firebase CLI Login

```

firebase login
firebase use attenglish-dev

```

Architecture

Feature-Based Architecture

Each feature module is self-contained:

```

feature/
├── models/      # Data classes (toJson, fromJson, fromFirestore)
├── screens/     # UI screens (StatefulWidget)
├── services/    # Business logic (Singleton pattern)
└── widgets/    # Reusable UI components for this feature

```

Benefits: - Features are independent and modular - Easy to locate code (everything for a feature is in one place) - Multiple developers can work on different features without conflicts - Easy to add or remove features

Services Layer

All services follow the **Singleton pattern**:

```
class LessonService {
  static final LessonService instance = LessonService._();
  LessonService._();

  // Methods...
}

// Usage:
LessonService.instance.createLesson(...);
```

Routing

Routes are defined in *main.dart* using named routes:

```
routes: {
  '/login': (context) => const LoginScreen(),
  '/signup': (context) => const SignUpScreen(),
  '/admin-signup': (context) => const AdminSignUpScreen(),
  '/home': (context) => const HomeScreen(),
  '/classrooms': (context) => const ClassroomManagementScreen(),
  '/admin': (context) => const AdminManagementScreen(),
  '/admin/students': (context) => const StudentManagementScreen(),
  '/lessons/create': (context) => const CreateLessonScreen(),
  '/lessons/list': (context) => const LessonListScreen(),
}
```

Navigation uses *Navigator.pushNamed()* and *Navigator.pop()*.

State Management

- **Local state:** *StatefulWidget* with *setState()*
- **Auth state:** *StreamBuilder* on *FirebaseAuth.authStateChanges*
- **Language state:** *StreamBuilder* on *LocalizationService.localeStream*
- **Data fetching:** *FutureBuilder* and *StreamBuilder* for Firestore queries

Note: The *flutter_riverpod* package is included in dependencies but not actively used. The app primarily uses *StatefulWidget*s and service singletons.

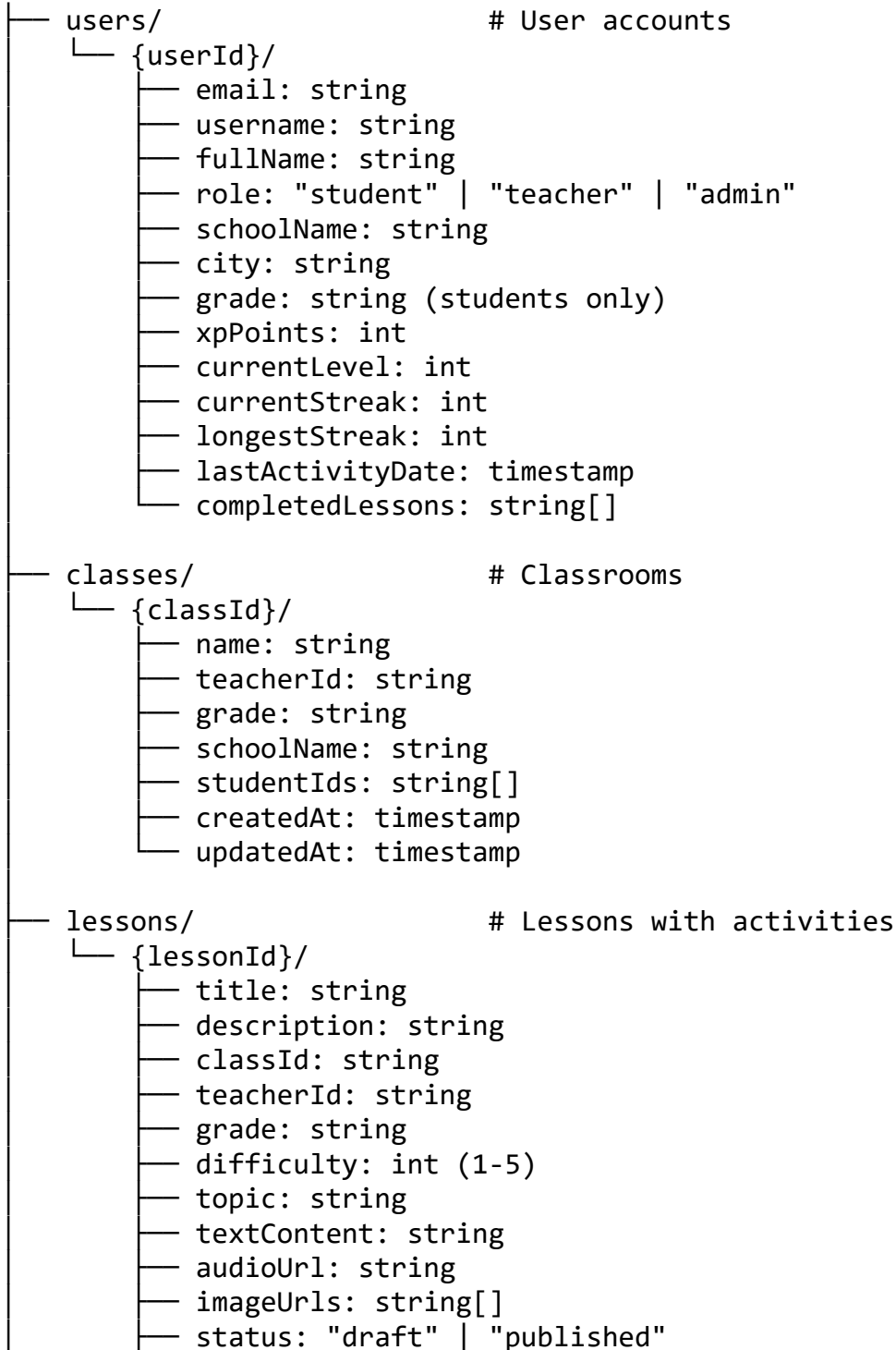
Firestore Configuration

Project Details

Property	Value
Project ID	<i>attenglish-dev</i>
Hosting URL	https://attenglish-dev.web.app
Console	https://console.firebase.google.com/project/attenglish-dev

Firestore Collections

Firestore Database



```

    |
    |— isAiGenerated: bool
    |— activities: Activity[] # Nested array
    |— createdAt: timestamp
    |— updatedAt: timestamp
  — lesson_completions/ # Student progress records
    |— {completionId}/
      |— lessonId: string
      |— studentId: string
      |— classId: string
      |— score: int
      |— maxScore: int
      |— completedAt: timestamp
  — student_progress/ # Additional progress data
    |— {progressId}/
      |— studentId: string
      |— lessonId: string
      |— ...

```

Firestore Auth

- Email/password authentication
- Custom user documents in *users* collection (linked by UID)
- Username-based login supported (looks up email from Firestore)

Firestore Storage

- Used for: lesson audio files, lesson images, uploaded documents
- Path format: *Lessons/{LessonId}/audio/*, *Lessons/{LessonId}/images/*

Cloud Functions

Location

```

functions/
|— src/
  |— index.ts # Main Cloud Function
|— package.json # Node.js dependencies
|— tsconfig.json # TypeScript config
|— lib/ # Compiled JavaScript (auto-generated)

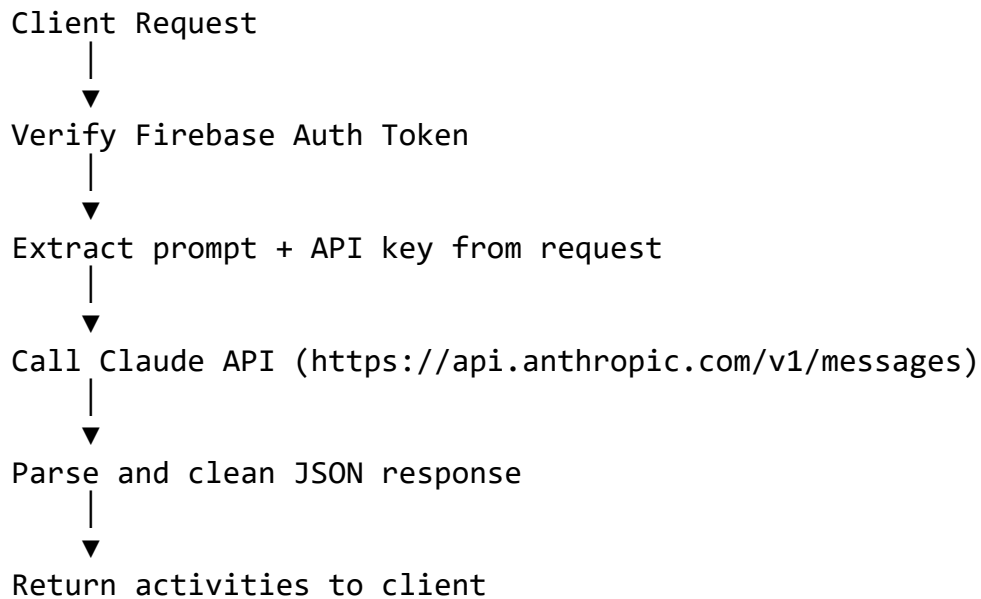
```

Function: *generateActivitiesWithAI*

Purpose: Proxy Claude API calls from the web client (avoids CORS issues and protects the API key).

Trigger: HTTPS Callable

Authentication: Firebase Auth ID token required

Flow:

Error Handling: - 429 (Rate Limited): Retry with exponential backoff (2s, 4s, 8s) - 503 (Overloaded): Same retry logic - 401/403: Authentication error returned to client - 500: Generic error with message

Configuration: - Timeout: 120 seconds - Memory: 512 MB - Runtime: Node.js 20

Cloud Functions Commands

`cd functions`

Install dependencies

`npm install`

Build (compile TypeScript)

`npm run build`

Test Locally with emulator

`npm run serve`

Deploy to Firebase

`npm run deploy`

or

`firebase deploy --only functions`

View Logs

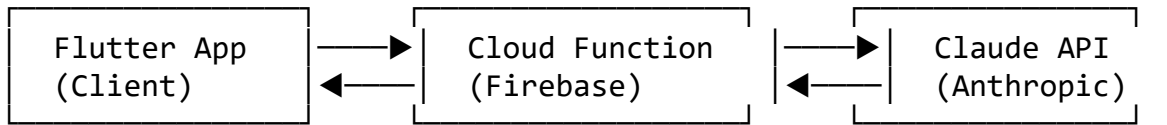
`npm run logs`

or

`firebase functions:log`

AI Integration

Architecture



Why Cloud Functions? The API key stays on the server, never exposed in client-side code.

Key Files

File	Purpose
<code>core/config/ai_config.dart</code>	API configuration (model, limits, URLs)
<code>features/ai_generation/services/claude_ai_service.dart</code>	Main AI service
<code>features/ai_generation/services/prompt_builder_service.dart</code>	Builds prompts for Claude
<code>features/ai_generation/services/document_processor_service.dart</code>	PDF/DOCX text extraction
<code>features/ai_generation/services/document_storage_service.dart</code>	Document upload to Firebase
<code>functions/src/index.ts</code>	Cloud Function proxy

AI Configuration (`ai_config.dart`)

```
apiBaseUrl:      'https://api.anthropic.com/v1'
modelName:       'claude-sonnet-4-20250514'
maxTokensPerRequest: 4096
requestTimeout:  60 seconds
maxWordsPerRequest: 5000
minWordsPerRequest: 50
```

Activity Generation Flow

1. Teacher pastes text (50-5,000 words)
2. `PromptBuilderService` builds a structured prompt including:
 - The source text
 - Grade level and difficulty
 - Requested activity types and counts
 - JSON output format specification
3. `ClaudeAiService` sends the request:
 - **Web:** Via Cloud Function (`_callClaudeViaCloudFunction`)
 - **Mobile/Desktop:** Direct API call (`_callClaudeApiDirect`)
4. Claude returns JSON with generated activities
5. `_parseActivities()` converts JSON to `ActivityModel` objects

6. Activities are displayed on the Review screen for teacher approval

AI Evaluation (Open-ended Questions)

For Image Description, Cloze, and Open-ended activity types: - Student submits a free-text answer - *ClaudeAiService.evaluateTextAnswer()* sends the answer + sample answer to Claude - Claude returns a score and feedback - Score is applied to the student's progress

Key Services Reference

AuthService (*features/auth/services/auth_service.dart*)

AuthService.instance

```
.signUpWithEmail(email, password, fullName, username, ...)
.signInWithEmail(emailOrUsername, password)
.signOut()
.updateUserProfile(userId, data)
.getUserDocument(userId) // Returns UserModel
```

LessonService (*features/lessons/services/lesson_service.dart*)

LessonService.instance

```
.createLesson(lesson)
.updateLesson(lessonId, data)
.deleteLesson(lessonId)
.getLesson(lessonId)
.getLessonsForClass(classId) // Returns Stream
.uploadFile(path, bytes)
.deleteFile(url)
```

ClassService (*features/classroom/services/class_service.dart*)

ClassService.instance

```
.createClass(classModel)
.updateClass(classId, data)
.deleteClass(classId)
.addStudentToClass(classId, studentId)
.removeStudentFromClass(classId, studentId)
.bulkImportStudents(classId, excelData)
```

ClaudeAiService (*features/ai_generation/services/claude_ai_service.dart*)

ClaudeAiService.instance

```
.generateActivities(text, config) // Returns List<ActivityModel>
>
.evaluateTextAnswer(question, answer) // Returns score + feedback
.evaluateImageDescription(imageUrls, answer)
```

ProgressService (*features/student_progress/services/progress_service.dart*)

ProgressService.instance

```
.recordLessonCompletion(studentId, lessonId, classId, score, maxScore)
```

```
.getStudentProgress(studentId, classId)
.getClassProgress(classId, {dateRange})
```

GamificationService

(features/gamification/services/gamification_service.dart)

GamificationService.instance

```
.addXP(userId, points)
.updateLevel(userId)
.updateStreak(userId)
.getAchievements(userId)
```

LocalizationService *(core/services/localization_service.dart)*

LocalizationService.instance

```
.initialize()
.setLocale(locale)
.currentLocale      // Getter
.localeStream       // Stream<Locale>
.textDirection      // LTR or RTL
```

Build & Deploy

Web Build

```
# Build for production
flutter build web --release
```

Output directory: build/web/

Deploy to Firebase Hosting

```
# Deploy web only
firebase deploy --only hosting
```

```
# Deploy Cloud Functions only
firebase deploy --only functions
```

```
# Deploy everything
firebase deploy --only hosting,functions
```

Hosting configuration *(firebase.json)*:

```
{
  "hosting": {
    "public": "attenglish/build/web",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"]
  }
}
```

Android Build

Build APK

```
flutter build apk --release
```

Output: *build/app/outputs/flutter-apk/app-release.apk*

Full Deployment Checklist

1. Check code quality

```
flutter analyze
```

2. Build web

```
flutter build web --release
```

3. Deploy web + functions

```
firebase deploy --only hosting,functions
```

4. Verify at <https://attenglish-dev.web.app>

Dependencies

Firestore

Package	Version	Purpose
<i>firebase_core</i>	^3.6.0	Firestore initialization
<i>firebase_auth</i>	^5.3.1	User authentication
<i>cloud_firestore</i>	^5.4.4	NoSQL database
<i>firebase_storage</i>	^12.3.4	File storage
<i>cloud_functions</i>	^5.1.3	Cloud Functions client

UI & Design

Package	Version	Purpose
<i>cupertino_icons</i>	^1.0.8	iOS-style icons
<i>getwidget</i>	^7.0.0	UI component library
<i>cached_network_image</i>	^3.4.1	Image caching and loading
<i>fl_chart</i>	^0.68.0	Charts for progress visualization

State & Navigation

Package	Version	Purpose
<i>flutter_riverpod</i>	^2.5.1	State management (imported, not heavily used)
<i>go_router</i>	^14.6.2	Navigation (imported, not actively used)
<i>shared_preferences</i>	^2.2.2	Local key-value storage

Document Processing

Package	Version	Purpose
<i>syncfusion_flutter_pdf</i>	^24.1.41	PDF text extraction
<i>archive</i>	^3.4.10	DOCX extraction (ZIP format)
<i>file_picker</i>	^8.1.6	File selection dialogs
<i>excel</i>	^4.0.6	Excel parsing for bulk student import
<i>csv</i>	^6.0.0	CSV handling
<i>spreadsheet_decoder</i>	^2.3.0	Spreadsheet reading
<i>desktop_drop</i>	^0.7.0	Drag-and-drop file upload (desktop)

Media & Network

Package	Version	Purpose
<i>audioplayers</i>	^6.1.0	Audio playback in lessons
<i>http</i>	^1.1.0	HTTP client for API calls
<i>url_launcher</i>	^6.2.2	Open external URLs
<i>flutter_dotenv</i>	^5.1.0	Load environment variables

Update Dependencies

Check for outdated packages

```
flutter pub outdated
```

Update to latest compatible versions

```
flutter pub upgrade
```

Install after pubspec.yaml changes

```
flutter pub get
```

Troubleshooting

Common Issues

Build Failures

Clean build artifacts

```
flutter clean
```

```
flutter pub get
```

```
flutter build web --release
```

Firebase Authentication Errors

- **“user-not-found”**: User doesn’t exist in Firebase Auth
- **“wrong-password”**: Incorrect password
- **“email-already-in-use”**: Email already registered
- Check Firebase Console > Authentication for user records

Firestore Permission Denied

- Check Firestore Security Rules in Firebase Console
- Ensure user is authenticated before making queries
- Verify the user's role matches the required permission

AI Generation Failures

- **429 Error:** Rate limited - wait and retry (auto-retry built in)
- **503 Error:** Claude API overloaded - wait and retry
- **Invalid API Key:** Check `.env` file, key should start with `sk-ant-`
- **Empty response:** Check prompt length (50-5,000 words)
- **JSON parse error:** Cloud Function cleans markdown blocks; check `functions/src/index.ts`

Cloud Functions Issues

View function logs

```
firebase functions:log
```

Redeploy functions

```
cd functions && npm run build && cd ..  
firebase deploy --only functions
```

Web-Specific Issues

- **CORS errors:** AI calls must go through Cloud Functions on web
- **Font warnings:** "Could not find Noto fonts" - non-blocking, can be ignored
- **Hot reload not working:** Stop and restart `flutter run -d chrome`

Useful Commands

Check Flutter environment

```
flutter doctor
```

Analyze code for issues

```
flutter analyze
```

Run tests

```
flutter test
```

List connected devices

```
flutter devices
```

Firebase project info

```
firebase projects:list
```

View Firestore data

Use Firebase Console: <https://console.firebase.google.com/project/attenglish-dev/firestore>

Performance Tips

- Use `StreamBuilder` for real-time data (classrooms, lessons)
- Use `FutureBuilder` for one-time data fetches
- Leverage `cached_network_image` for all remote images

- Keep Firestore queries scoped (filter by classId, teacherId)
- Use `.Limit()` on Firestore queries for large collections